

Stage 1 Lab - Human vs AI: Building Your First SmartCare Prototype

Learning objectives

- Create and run a simple Python file with basic input,output and processing statements
- Use lists, dictionaries and functions to enhance the Python file
- Build a small SmartCare appointment prototype.
- Use AI as a tutor rather than a replacement.
- Compare human-written and AI-generated code.
- Verify AI-generated code through execution and test inputs.
- Document a short AI-use reflection.

Files to create and commit in GitHub

stage01/
smartcare_v01.py
comparison.md
reflection.md
ai_usage.md

Part A - Understand the Problem: AI OFF

SmartCare needs a small prototype that allows a receptionist to record patient appointments. Each appointment records patient name, practitioner name and appointment time.

What data must be stored?

- Patient name
- Appointment data
- Appointment date
- Appointment time
- Name of practitioner
- Patient files
- Patient previous history

What functions might be useful?

- Book an appointment
- Cancel an appointment
- View an appointment
- Search for an appointment
- Search for a patient
- Check with an appointment time is already booked
- Check to see if a practitioner is already booked

- Check previous practitioner

What could go wrong?

- Two patients could be booked at the same time
- Incorrect patient or practitioner could be entered
- Incorrect date or time could be entered
- An appointment could be recorded more than once
- The appointment might not save
- Important information might be left blank

What requirements are unclear?

- Should the system allow appointments to be cancelled or changed?
- Should it store patient details?
- Can patients book their own appointments?
- Should double-bookings be prevented by the system automatically?

Part B - Build a Human-Written Prototype: AI OFF

#task1

Create and run a simple Python file with basic input,output statements

```
print("Welcome to SmartCare: Community Clinic Appointment Booking System!")
```

First Appointment

```
patient1_name = 'Alice Smith'
```

```
practitioner1_name = 'Dr. John Doe'
```

```
appointment1_time = '2024-07-20 10:00 AM'
```

```
print(f"Patient: {patient1_name} | Practitioner: {practitioner1_name} | Time: {appointment1_time}")
```

Second Appointment

```
patient2_name = 'Bob Johnson'
```

```
practitioner2_name = 'Dr. Jane Roe'
```

```
appointment2_time = '2024-07-20 11:30 AM'
```

```
print(f"Patient: {patient2_name} | Practitioner: {practitioner2_name} | Time: {appointment2_time}")
```

#task1enhanced

Use lists, dictionaries and functions to enhance the Python file

```
appointments = []
```

```
def book_appointment(patient_name, practitioner_name, appointment_time):
```

```
    if not patient_name:
```

```
        raise ValueError("Patient name cannot be empty")
```

```

    appointment = {
        "patient": patient_name,
        "practitioner": practitioner_name,
        "time": appointment_time
    }
    appointments.append(appointment)

def display_appointments():
    if not appointments:
        print("No appointments recorded.")
        return
    for appointment in appointments:
        print(f"Patient: {appointment['patient']} | Practitioner:
{appointment['practitioner']} | Time: {appointment['time']}")

print("Welcome to SmartCare: The Clinical Appointment Booking System!")
book_appointment('Alice Smith', 'Dr. John Doe', '2024-07-20 10:00 AM')
book_appointment('Bob Johnson', 'Dr. Jane Roe', '2024-07-20 11:30 AM')
display_appointments()

```

^ Now, run both programs , and identify at least five limitations.

- It is hardcoded, so the staff member can't change details as the appointment is being booked
- There is currently no way to cancel an appointment
- It is messy, for multiple appointments on different days, it is hard to keep track of
- The system currently doesn't check if that time/date slot is already taken
- The receptionist has to rely on the code rather than interacting with a simple menu.

Part C - Use AI as Tutor: AI ON (Use only UC approved GenAI Tool such as Microsoft CoPilot)

Suggested prompt structure:

Act as a Python tutor.

I am learning introductory software technology.

Here is a small appointment-booking function.

1. Explain what the code does.
2. Identify three limitations.
3. Suggest improvements.
4. Do not rewrite the whole application.
5. Ask me two questions to test my understanding.

Part D - Generate an Alternative: AI ON

Ask AI to create a simple beginner-friendly Python function that stores patient name, practitioner name and appointment time. Explicitly prohibit a database or GUI.

AI Code:

```
def book_appointment(patient_name, practitioner_name, appointment_time):

    appointment = {

        "patient": patient_name,

        "practitioner": practitioner_name,

        "time": appointment_time

    }

    return appointment

appointment1 = book_appointment(

    "Alice Smith",

    "Dr. John Doe",

    "2024-07-20 10:00 AM"

)

print(appointment1)
```

Part E - Compare Human and AI Versions

Question	Human version	AI version
Easy to understand?	Somewhat	No
Runs successfully?	Yes	No
Uses only required features?	Yes	Yes
Adds assumptions?	No	No
Handles errors?	Somewhat	No
Could I explain it?	Yes	Somewhat

Part F - Verify Behaviour

- Normal appointment **Yes**
- Blank patient name **No**
- Two appointments for the same practitioner/time **No**
- Strange input such as patient_name=None or appointment_time=None **Yes**

Part G - Improve One Thing

Choose exactly one controlled improvement, for example: if not patient_name: raise ValueError("Patient name cannot be empty")

```
for existing in appointments: if (existing["practitioner"] == practitioner_name and existing["time"] == appointment_time): raise ValueError("Practitioner is already booked at this time")
```

I added a way to determine if an appointment slot was already taken, as this will prevent double bookings.

Part H - Reflection (150-250 words)

What did you build before using AI?

Before using AI I had built a model that was able to book appointments. It stored patient names, practitioner names and appointment times using variables. I then enhanced it using a list, dictionaries and functions to record and display multiple appointments. The prototype demonstrated basic appointment management but had several limitations.

What did AI help you understand?

AI helped me understand how lists, functions and dictionaries can be used to help organise data. It had also helped me determine any errors in my code and the way to go about fixing these errors. Most importantly I learnt that AI code doesn't always work the way you imagined and will often make mistakes that are hard to pick up.

Did AI make assumptions?

AI didn't make any assumptions in my case, I would assume this is because only three variables were presented to the AI and so didn't feel it was necessary to overcomplicate the code. Nothing outside the requirements were added.

How did you verify the AI output?

I ran the AI code in the VS Code interface. I tested a normal appointment, a blank patient name, two appointments for the same practitioner and time and none values. I compared the actual results with the expected behaviour to identify whether the function worked correctly.

What engineering work remained for you?

The engineering work that remained was to test, evaluate and improve the AI-generated code. I needed to identify limitations, verify its behaviour with different inputs, check that it met SmartCare's requirements, and make a controlled improvement to address one specific problem.

Submission checklist [GitHub Commit]

- Python file runs.
- Comparison table completed.
- Normal and unusual inputs tested.
- AI assistance documented.

- Reflection completed.
- I can explain my code.